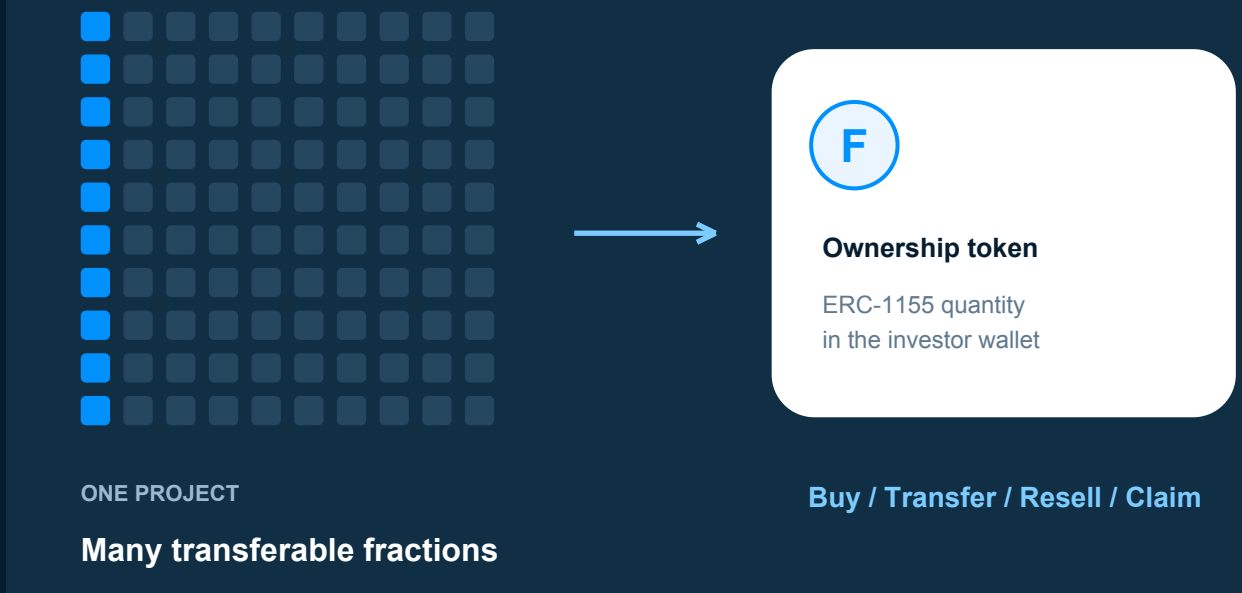
INFRA ONLY

reSystem Investment

The platform blueprint

Fractional ownership. Transferable rights.
Revenue funded and claimed on-chain.



Client planning document

16 October 2026 | Version 1.1
Business rules, system design, delivery phases and acceptance criteria.

Proposed architecture. Illustrations use examples, not investment performance.

01 / BUSINESS MODEL

What the platform does

A single INFRA platform for publishing projects, buying fractions, transferring ownership, reselling and claiming funded revenue.

Area	Agreed rule
Project publication	Images, documents, project value, fraction count and manual indicative yield.
Payment	Accept reAED or USDT directly. Fixed internal pricing: 1 USDT = 3.6 reAED.
Ownership	Each project has a separate contract. Whole ERC-1155 units represent fractions.
Participation	Any INFRA wallet may receive or trade fractions. Investor connection in the app uses reID.
Transfers and resale	Transfer any whole quantity. Marketplace supports partial fills in the first release.
Revenue	Only sold fractions share a funded allocation. Previously allocated rewards stay with the previous holder.
Administration	Username/password account, role permission and an authorized signing wallet.
Infrastructure	Separate repositories and deployments for each frontend, backend, contracts and indexer.

The fixed rate is a protocol pricing rule. It is not an automatic asset swap or a guarantee of external redemption.

The first release must keep these actions distinct

Buy fractions invests principal. Fund revenue creates claimable rewards. Sell fractions finds another buyer. Collect proceeds receives resale funds. Send wallet funds transfers assets to another INFRA address.

02 / FRACTION ECONOMICS

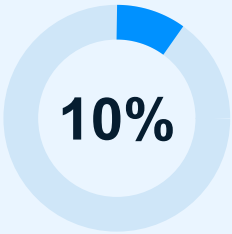
One project, two percentages

Project ownership and participation in the next revenue allocation use different denominators.

Illustrative setting	Value
Total project value	100,000 USDT / 360,000 reAED
Maximum fractions	10,000
Price per fraction	10 USDT / 36 reAED
Fractions sold	2,000
Investor holding	1,000

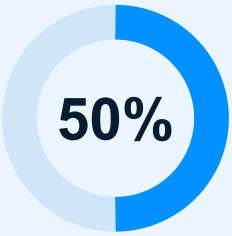
Project ownership

1,000 held / 10,000 maximum



Next revenue allocation

1,000 held / 2,000 sold



Why this matters: when more fractions sell, an existing investor keeps the same project ownership percentage. Their share of future revenue allocations changes as the sold-fraction denominator grows.

Publishing validates exact unit pricing. Maximum supply and initial economic terms become fixed when published. Token rights and any off-chain project obligations must be defined in the project documents.

Illustrative arithmetic only. These charts are not forecasts or yield estimates.

03 / INVESTOR JOURNEY

From project discovery to ownership

The wallet approves financial actions. Successful contract execution is the source of truth.

01

Discover

Read published terms, value, supply, documents and indicative yield.

02

Connect reID

Confirm the actual INFRA wallet and signing permissions.

03

Choose fractions

Select a whole quantity and reAED or USDT. Read the exact quote.

04

Approve and buy

USDT may require token approval before the purchase transaction.

05

Receive ownership

Payment and fraction minting complete atomically or revert together.

Contract checks: valid quantity, remaining supply, verified payment asset, exact amount received, current terms and successful receiver handling. Sale proceeds are kept separate from reward reserves.

Honest transaction status

Awaiting approval > **Broadcast** > **Confirmed** > **Indexed**

A transaction hash alone is not success. Rejected, expired, failed and delayed-indexing states each need a clear recovery path.

04 / REVENUE AND CLAIMS

Earlier rewards stay with the holder

Allocation occurs at the revenue funding transaction. Future allocations follow the latest fraction balances.

1. Owner funds the pool

A real reAED or USDT deposit is allocated to the sold fractions at that transaction.

Before transfer

Alice owns 1,000 fractions. Her existing reward entitlement is recorded.

Transfer 10 fractions

Alice sends 10 whole units to Bob. The transfer settles historical entitlement first.

After transfer: Alice

990 fractions earn future allocations.
All of Alice's earlier allocated rewards remain hers.

After transfer: Bob

10 fractions earn future allocations.
Bob does not receive Alice's earlier rewards.

- Claims pay the same asset that funded the allocation: reAED or USDT.
- A holder can claim earlier rewards after selling or transferring every fraction.
- New purchases never inherit historical allocations. Unsold fractions earn nothing.

Manual yield is descriptive. The owner enters an indicative yield, period and effective date. That number creates no claimable funds; only actual funded allocations do.

Accounting uses high-precision cumulative rewards per fraction, asset-specific balances and explicit rounding reserves. Deposits do not loop through all holders. Funding is rejected if no fractions have sold.

05 / RESALE MARKETPLACE

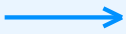
Sell a portion, keep the rest

One shared on-chain marketplace supports registered projects and partial fills in the first release.

SELLER LISTS

1,000 fractions

Unit price + payment asset + expiry



Buyer purchases

10 fractions

990 remain listed

Step	Required behavior
Create listing	Seller selects project, quantity, unit price, settlement asset and expiry, then grants approval.
Keep ownership	Fractions stay in the seller wallet until filled. The seller earns allocations while holding them.
Validate fill	Check listing, expiry, quantity, balance, approval, price, asset and registered project.
Settle atomically	Collect buyer payment and transfer fractions together, or revert the complete trade.
Collect proceeds	Credit a separate on-chain seller proceeds balance. Seller withdraws these funds.
Cancel or invalidate	Seller can cancel. Missing balance, revoked approval or expiry makes a listing unavailable.

No guaranteed buyer or buyback. A listing does not promise an instant sale or recovery of the original investment. Proposed launch marketplace fee: zero.

A seller's historical project rewards are separate from resale proceeds. The UI must show each balance and collection action clearly.

06 / SYSTEM ARCHITECTURE

Blockchain settles. Services organize.

Economic rights and asset movements live on INFRA. Off-chain services make the platform searchable, manageable and reliable.



Trust boundary: the database cannot create ownership or rewards. Contract balances, funded allocations and confirmed receipts remain authoritative. Indexer projections are rebuildable.

Target hosting, not a deployment claim. FP services use Teleport for server access. Project documents define off-chain rights; contracts cannot prove that all external revenue was deposited.

07 / CONTRACT INVENTORY

The on-chain function map

Proposed interfaces. Final ABIs follow the detailed specification and INFRA compatibility checks.

Contract / area	Functions and responsibilities
Master factory	createProject; getProject; isRegisteredProject; implementation version registry; authorized creation; pauseCreation.
Project configuration	Read value, maximum fractions, sold fractions, owner, treasury, prices, metadata URI and sale state.
Primary sale	quotePurchase; buyWithReAED; buyWithUSDT; openSale; closeSale; withdrawSaleProceeds.
ERC-1155 ownership	balanceOf; safeTransferFrom; safeBatchTransferFrom; setApprovalForAll. Reward settlement on balance changes.
Revenue pool	depositRevenueReAED; depositRevenueUSDT; claimable; claim; claimBoth; claimFor with fixed entitled recipient.
Project presentation	setIndicativeYield; metadata revisions; freezeMetadata. Record version and effective date.
Marketplace	createListing; cancelListing; getListing; quoteFill; fillWithReAED; fillWithUSDT; withdrawProceeds.
Claim router	Bounded claims across registered projects. Each payment goes to the entitled wallet.
Access and recovery	Explicit roles; two-step ownership transfer; scoped pause controls; no reward-reserve withdrawal or unlimited mint.

Recommended deployment model: ERC-1167 clones with fixed implementation versions. Every project receives a separate address. New versions serve new projects; existing projects do not change silently.

Event coverage: project creation, purchases, ownership transfers, sale changes, revenue funding, claims, yield and metadata changes, listings, fills, cancellations, proceeds withdrawals and role changes.

08 / ADMIN APPLICATION

Controlled operations, visible accountability

Application authorization and blockchain signing authority are independent requirements.

Account session + Required permission + Authorized wallet

Role	Primary permissions
Super admin	Staff, roles and platform configuration.
Project manager	Draft content, media, review and publication workflow.
Project owner	Assigned projects, indicative yield and authorized project operations.
Finance preparer	Prepare revenue funding and treasury requests.
Finance approver	Approve and sign permitted finance actions.
Support	Ticket handling and limited investor context.
Auditor	Read-only records, transactions and reports.
Operator	Indexer health, recovery and backfills.

Admin modules

Dashboard and alerts; project creation wizard; preview and publication; documents and metadata; holders and primary sales; revenue and claims; resale listings and proceeds; investor records; staff and wallet bindings; support inbox; audit logs; chain health.

Username/password uses Argon2id, secure sessions, rate limiting and privileged-user MFA. MetaMask or reID supplies the signing wallet. Connecting a wallet does not create an admin role.

Treasury and contract administration should use an INFRA-compatible multisignature arrangement after compatibility verification.

09 / REPOSITORIES AND TECHNOLOGY

Five independent codebases

Each repository has its own CI, release process and deployment. Shared interfaces are versioned artifacts.

Repository	Technology / responsibility	Target hosting
Investor frontend **Existing investment repo**	Next.js, React, TypeScript; public site and investor app.	Vercel
Admin frontend	Independent Next.js app; staff operations and signing.	FP server
Backend	NestJS / TypeScript, PostgreSQL, Prisma; auth, workflows, APIs and reports.	FP server
Smart contracts	Solidity, OpenZeppelin, Foundry; factory, projects, marketplace, tests, deployment manifests and ABIs.	INFRA; current integration is testnet
Standalone indexer	TypeScript and viem; ingestion, confirmations, reconciliation and backfills.	FP server

Supporting tools

- **viem:** reads, calldata, transaction simulation and receipts.
- **reID SDK:** investor connection and transaction approval. **MetaMask:** admin signing.
- **Pinata / IPFS:** public images, NFT metadata and published documents.
- **PostgreSQL + Redis / BullMQ workers:** FP server. **Docker:** service packaging.
- **GitHub Actions:** CI and releases. **Vitest / Playwright:** application checks.

Confirmed target: the investor frontend uses Vercel. Admin, backend, database, worker and indexer services use the FP server, accessed through Teleport. New services are not claimed as deployed.

API choice: REST / OpenAPI remains the default. GraphQL is optional and, if adopted, runs on the FP server. Use versioned generated clients and contract artifacts; avoid cross-repository relative imports.

Pin compiler, libraries and the EVM target after checking INFRA compatibility.

10 / BACKEND AND INDEXING

Live data with recoverable operations

Services support identity, workflow and search. They never substitute a database record for a confirmed blockchain action.

Backend module	Required functions
Identity and connections	reID callbacks; encrypted sessions and tokens; refresh rotation; scopes; reconnect and logout.
Admin and authorization	Username/password, MFA, invites, recovery, roles, project scope, wallet binding and suspension.
Projects	Draft, validate, media/documents, review, publish, revisions and archive.
Transactions	Validated commands, quotes, simulation, gas checks, intents, approval status, receipts and idempotency.
Read models	Portfolio, balances, ownership, activity, revenue, claims, listings, fills and proceeds.
Operations	Tickets, notifications, audit, reconciliation, backfills and deployment registry.

Indexer guarantees

Rebuildable history

Durable block cursors, deduplicated events, block hashes, reorganization recovery and ordered processing.

Operational clarity

Confirmation policy, retries, backfills, restart safety, reconciliation and visible indexing lag.

reID work still required: the existing sign-in uses read permissions. Production signing needs wallet:tx consent, encrypted connection storage, token rotation, approval links, polling and receipt tracking.

The backend constructs transactions from validated commands. It checks chain, project, function, quantity, asset, recipient and price. It must not expose an unrestricted arbitrary-calldata signing endpoint.

Wallet deposits need native and ERC-20 activity ingestion. An incoming wallet transfer does not automatically create a purchase or a project revenue allocation. Use integer base units throughout.

11 / INVESTOR SCREENS

Turn the existing UI into a live product

Keep the established branding and responsive layouts while replacing simulated state with verified records.

Screen	Production behavior
Homepage and explore	Published projects, real availability, search/filter and manually entered indicative yield.
Project details	Value, unit prices, supply, documents, pool activity and purchases.
Dashboard and portfolio	Actual holdings, ownership percentages, balances, claims and activity.
Earnings	Funded allocations, per-asset history and real claim transactions.
Wallet and deposit	Actual reAED, USDT and ERC-1155 balances; user address, QR and incoming transfers.
Fraction transfer	Select project, recipient and whole quantity; review and approve.
Marketplace	Browse, list, partially buy, cancel and collect seller proceeds.
Withdraw / send	Send wallet assets to an INFRA address while reserving native gas.
Activity	Purchases, transfers, fills, claims and wallet transactions with confirmation status.
Support / notifications	Persistent tickets, replies, account events and read/unread state.

Current baseline: reID sign-in and the deposit receiving flow have real integrations. The broader investment UI is still a prototype. Balances, purchases, claims, resale and related activity need production contracts and services.

Remove hard-coded prices, contradictory supplies, sample transactions, fake live activity and simulated credits. Bank transfer and card stay disabled with Coming soon treatment.

12 / DELIVERY AND ACCEPTANCE

Build in dependency order

No deadline or budget is assumed. Each phase has an explicit completion gate.

1. **Specify**
   Finalize economic rules, permissions, interfaces and transaction state machines.

2. **Prove integrations**
   Verify INFRA, reID signing, MetaMask, USDT approval and ERC-1155 receipt.

3. **Build contracts**
   Implement and test the factory, project, marketplace and claim router.

4. **Build services**
   Persistent auth, API, indexer, reconciliation and generated clients.

5. **Build admin**
   Project publication, roles, wallet signing and revenue funding.

6. **Make investor UI live**
   Purchases, transfers, partial resale, claims, balances and activity.

7. **Rehearse on UAT**
   Complete multi-wallet lifecycle, failures, indexing recovery and role checks.

8. **Release**
   Security review, verified deployments, role setup, monitoring and configuration.

Required release rehearsal

Create project → publish → buy in both assets → buy more → transfer 10 fractions → partially resell → fund revenue → claim → fully exit → claim historical rewards.

Contract verification combines unit, fuzz and invariant tests with static analysis. Application verification covers API authorization, real receipts, indexed state, mobile layouts and failure recovery.

13 / FINAL GATES AND REFERENCES

What must be true before launch

These rules are acceptance criteria, not optional implementation details.

- Supply never exceeds the fixed maximum. Transfers and resale never create fractions.
- New buyers receive no old allocations; previous holders retain their settled rewards.
- Claims never exceed funded revenue; treasury actions cannot consume reward reserves.
- Partial fills cannot oversell. Failed payment or receiver handling reverts the complete trade.
- All financial actions require wallet authorization and the correct INFRA network.

Proposed defaults to confirm: zero platform fees; allocation at the funding transaction; versioned fixed implementations; no guaranteed platform buyback. Revenue participation is not time-weighted across a reporting period.

Deployment prerequisites

Verify the intended INFRA environment, RPC behavior, USDT address and decimals, gas rules, finality policy, explorer support and multisignature compatibility. The current integration uses INFRA UAT chain ID 15324; do not assume that this is the final production configuration.

Development credentials are held in a dedicated private operational Notion page. Rotate them before production. This PDF contains no credential values, private keys, access tokens or real investor wallet data.

Technical references

ERC-1155 ownership standard

ERC-1167 minimal proxy standard

OpenZeppelin ERC-1155 implementation

OpenZeppelin ERC-20 utilities

Pinata upload documentation

viem contract simulation

Foundry invariant testing

OWASP password storage guidance

Planning baseline: the agreed requirements and platform plan recorded on 16 October 2026. Detailed implementation and test evidence are tracked separately in the project task system.